


**SHARIF UNIVERSITY OF TECHNOLOGY**

Department of Electrical Engineering


TECHNICAL REPORT

# Coverage-Driven Verification of a Multi-Cycle ALU


AUTHOR

Reza Sanaee Nezhad

Student ID: 401101502

INSTRUCTOR

Not identified in the supplied materials

# Contents

1 Scope and verification architecture 2

2 Parameters and operation contract 3

3 Input and output interfaces 4

4 Input transactions, constraints and copying 6

5 Output transactions and interruption state 8

6 Driver scheduling and packet serialization 8

7 Input monitor and observed idle gap 8

8 Output monitor and reset event capture 9

9 Reference model 9

10 Scoreboard ordering and legal cancellation 11

11 Testbench integration and test profiles 11

12 Functional coverage and complete test plan 13

13 Recorded academic coverage and ranking figures 14

14 Reproducible simulation results 16

15 Execution and engineering limits 17

A Implementation details 18

# 1 Scope and verification architecture

This project develops a class-based SystemVerilog verification environment for a supplied multi-cycle arithmetic logic unit. It adapts interfaces, transactions, driver, monitors, reference model, scoreboard, coverage and test selection to a packetized operand/result protocol. The environment uses mailboxes and events rather than the UVM library. The supplied RTL and framework headers credit Alireza Abbasian; the verification project and its report are by Reza Sanaee Nezhad.

An input transaction represents a complete operation and two full operands. The driver transmits that transaction as a sequence of bus-width chunks. The input monitor reconstructs accepted chunks independently and passes the complete transaction to the reference model. The output monitor collects result chunks or reset notifications. The scoreboard checks them against the reference queue and distinguishes completed results from operations canceled by a legal reset. Functional covergroups connect the tests to a spreadsheet-style plan.

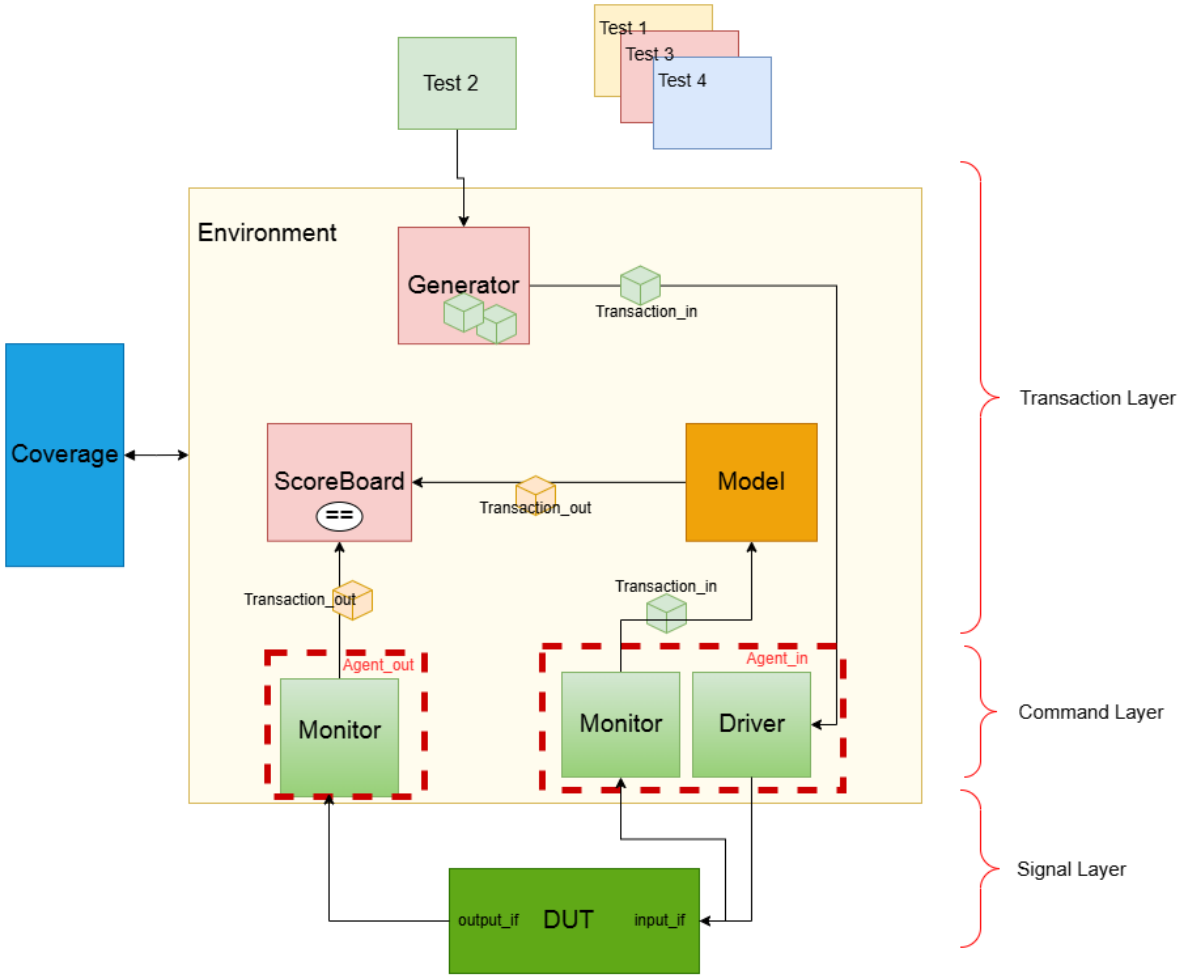



Figure 1: Supplied verification architecture: generator, driver, input/output monitors, reference model, scoreboard and coverage.

## 2 Parameters and operation contract

The default operand bus width is eight bits and each complete unsigned operand is 32 bits. The result bus is 16 bits and complete result storage is 64 bits. A complete operand pair therefore uses four accepted input beats. Results are sent least-significant chunk first, using one to four beats as indicated by result_last. A zero or sufficiently small result uses one beat. The monitor reconstructs the complete result in initially zeroed storage, so omitted leading zero chunks do not leave stale data.

The physical rst input is active low. Software reset is opcode 111 with operand_valid asserted, and it is accepted even while ready is low. The normal opcodes are no-op 000, unsigned addition 001, bitwise AND 010, bitwise XOR 011 and unsigned multiplication 100. Opcodes 101 and 110 are excluded by the transaction constraint and classified as illegal coverage bins. No-op does not generate an arithmetic output transaction. Software reset clears result storage, deasserts result_valid, asserts result_rst and returns the unit to IDLE.

The RTL uses IDLE, RECEIVE, EXECUTE and SEND states. ready is asserted in IDLE and RECEIVE. Arithmetic uses the operation captured with the first operand beat. Operand widths must divide the full operand width, result bus width must divide the full result width, and the result width must accommodate the complete unsigned product. Parameter checks reject unsupported configurations. The result bus has one combinational writer driven from the full-result register and chunk index.

Table 1: Legal operation semantics

| **Opcode** | **Operation** | **Output reference** |
|---|---|---|
| 000 | No-op | No arithmetic result |
| 001 | Unsigned add | Extended a+b |
| 010 | Bitwise AND | a & b |
| 011 | Bitwise XOR | a XOR b |
| 100 | Unsigned multiply | Full a*b |
| 111 | Software reset | Zero; result_rst=1 |

# 3 Input and output interfaces

The input interface carries ready, operand_valid, operand_last, op, a and b together with clock and active-low reset. Its DUT modport receives the operand and control signals and produces ready. The driver modport drives operands and control while observing ready, clock and reset. The monitor modport observes all signals without driving them. Distinguishing these directions makes the interaction between the three roles explicit.

The output interface carries result_valid, result_last, result_rst and the result chunk. The DUT drives these signals, and the monitor observes them. The top-level module connects the four width parameters and every input/output control signal to the corresponding DUT port. Both interfaces share the same system clock and physical reset.

```
import parameters::*;

interface ALU_in_if (
    input logic                 clk,
    input logic                 rst // active low
);

    logic                   ready;
    logic                   operand_valid;
    logic                   operand_last;
    logic [2:0]             op;
    logic [OPERAND_BUS_WIDTH-1:0]    a;
    logic [OPERAND_BUS_WIDTH-1:0]    b;


    modport dut_mp (
        input clk,
        input rst,
        output ready,
        input operand_valid,
        input operand_last,
        input op,
        input a,
        input b
    );

    modport driver_mp (
        input clk,
        input rst,
        input ready,
        output operand_valid,
        output operand_last,
        output op,
        output a,
        output b
    );

    modport monitor_mp (
        input clk,
        input rst,
        input ready,
        input operand_valid,
        input operand_last,
        input op,
        input a,
        input b
    );


endinterface
```

Figure 2: Input interface and direction-specific modports.

```
import parameters::*;


interface ALU_out_if (
    input logic                 clk,
    input logic                 rst
);

    logic                       result_valid;
    logic                       result_last;
    logic                       result_rst;
    logic [RESULT_BUS_WIDTH-1:0]    result;


    modport dut_mp (
        input clk,
        input rst,
        output result_valid,
        output result_last,
        output result_rst,
        output result
    );

    modport monitor_mp (
        input clk,
        input rst,
        input result_valid,
        input result_last,
        input result_rst,
        input result
    );


endinterface


```

Figure 3: Output interface and observer modport.

## 4 Input transactions, constraints and copying

ALU_in_transaction contains a randomized operation, full-width a and b values, and the four-bit randomized clk_gap. Its legal-operation constraint permits the six defined opcodes. Its gap constraint permits values zero through fourteen inclusive. clk_gap specifies ready idle cycles before starting a normal operand frame; it does not represent the total latency of a previous operation or clocks spent waiting for ready.

The input transaction stores all 32 bits of each operand even though the bus carries eight bits at a time. Serialization belongs to the driver rather than to the generator. do_copy copies op, a, b and clk_gap into a separate object. do_compare compares operation and operand values; timing gap is excluded because two transactions with identical arithmetic inputs can have different idle spacing without changing the mathematical result.

ALU_in_transaction_range extends the base transaction with inclusive minimum and maximum bounds for each operand. Randomization obeys both the inherited legal-operation/gap constraints and the range constraints. Its copy method retains the range bounds and clk_gap as well as arithmetic fields. Range display uses inclusive endpoints, matching the actual greater-than-or-equal and less-than-or-equal constraints.

```
1  lass ALU_in_transaction;
2
3  //-------------------The random variables
4      rand alu_in_op_t                    op;
5      rand bit           [OPERAND_MAX_DATA_WIDTH-1:0]   a;
6      rand bit           [OPERAND_MAX_DATA_WIDTH-1:0]   b;
7      rand bit           [3:0]   clk_gap;
8  //---------------------------------------
9      time issue_time;
10
11 //---------------------------constraints
12
13     constraint valid_op_c {
14         op inside {no_op, add_op, and_op, xor_op, mul_op, rst_op};
15     }
16
17     constraint valid_clk_gap {
18         clk_gap<4'd15;
19         clk_gap>=4'd0;
20     }
```

Figure 4: Full-width randomized operands and constrained idle gap.

```
virtual function ALU_in_transaction do_copy();
    ALU_in_transaction dst;

    dst = new();

    dst.op  = this.op;
    dst.a   = this.a;
    dst.b   = this.b;
    dst.clk_gap=this.clk_gap;

    return dst;
```

Figure 5: Input transaction copying includes clk_gap.

## 5 Output transactions and interruption state

ALU_out_transaction stores the complete 64-bit result, result_rst, an aborted flag and an issue timestamp. A new object initializes interruption state to false, and complete reconstruction starts with zeroed result storage. The copy method retains result, reset flag and interruption state in a separate object. Its arithmetic comparison checks result and result_rst.

The aborted flag means a pending arithmetic operation was interrupted by a software reset. It is not an alternative successful arithmetic result. In that event the observable reset output must be zero and must have result_rst asserted. The canceled arithmetic operation and the completed reset command are accounted for separately. This prevents a flagged result from being accepted merely because its interruption bit is set.

## 6 Driver scheduling and packet serialization

The generator sends transaction objects through gen2drv. The driver removes one object, transmits it, and signals drv2gen so that generation can continue. Stimulus is driven on falling clock edges and accepted by the DUT on rising edges. This places values on the bus before the DUT's sampling edge rather than changing them in the same active event region as DUT sampling.

For a normal transaction, the driver waits until the physical reset is released and ready is asserted. It waits the requested number of ready idle clocks, then sends four successive eight-bit portions from a and b. The indexed slice for beat i begins at bit i*OPERAND_BUS_WIDTH. operand_valid is asserted for each beat, and operand_last is asserted only for the final beat. After the last accepted beat, the control signals are deasserted on a falling edge. The full operand object is not modified by serialization.

For software reset, the driver waits for physical reset release but does not wait for ready. It asserts operand_valid with rst_op for one accepted cycle. This enables a reset command during calculation or result collection, when the unit is busy. Consecutive software-reset commands remain separate accepted commands even when result_rst is asserted in consecutive cycles. Generator/driver event synchronization prevents two threads from driving competing operand frames.

## 7 Input monitor and observed idle gap

The input monitor samples at rising edges, when the driver values are stable. An ordinary accepted beat requires operand_valid and ready while physical reset is released. A reset command can be recognized with operand_valid and rst_op even when ready is low. The monitor creates a new transaction at the start of an accepted frame and records the operation from the bus rather than from the driver's object.

For an ordinary frame, each accepted eight-bit a and b chunk is placed at its corresponding position in the 32-bit transaction. The monitor continues until operand_last and checks that the frame cannot exceed the declared maximum number of beats. A gap between beats is not filled with unaccepted bus data. The reconstructed object is copied into the model mailbox only after the frame has completed.

Idle-cycle counting includes ready clocks without a valid new operand, and excludes busy clocks and physical-reset time. The observed gap is checked against the zero-to-fourteen stimulus contract before assignment to the four-bit field. A software-reset input produces a reset transaction directly. Input functional coverage is sampled from monitored transactions, so it reflects accepted bus behavior rather than requested generator values.

# 8 Output monitor and reset event capture

The output monitor samples on falling edges, after the DUT's rising-edge updates have settled. It waits until the physical reset is released and either result_valid or result_rst is asserted. A result transaction is reconstructed from successive 16-bit chunks in 64-bit storage. result_last identifies the final chunk, and the monitor checks the maximum beat count.

If result_valid disappears before result_last without a reset notification, the monitor reports a protocol failure. If result_rst arrives during reconstruction, the partial numeric value is not treated as a valid completed answer; the record contains a zero reset result and is marked as interrupted. A reset arriving before the first result beat is also emitted as a reset record and is associated with the pending arithmetic reference by the scoreboard.

One software-reset record is emitted per active reset cycle. The next monitor call starts at the next falling edge. This allows two back-to-back accepted reset commands to be observed separately. Physical-reset cycles are excluded while rst is low. The stream is therefore not collapsed into one event merely because result_rst remains high across adjacent command cycles.

# 9 Reference model

The model receives independently monitored full input transactions. Addition produces an unsigned extended sum in result-width storage, AND and XOR produce their bitwise results, and multiplication produces the complete unsigned product. Ordinary arithmetic results have result_rst equal to zero. rst_op produces a zero result with result_rst equal to one. No-op produces no expected arithmetic result and is not enqueued into the output reference stream.

The model does not use the randomized idle gap in calculation. Time spacing influences protocol execution and coverage, but not arithmetic. A mailbox carries expected result objects to the scoreboard in accepted transaction order. The operation/result mapping is shown in the table and reference-model implementation.

```
function ALU_out_transaction calc(ALU_in_transaction tr_in);

    ALU_out_transaction tr_out;
    tr_out = new();

    case (tr_in.op)
        add_op: begin
            tr_out.result = tr_in.a + tr_in.b;
            tr_out.result_rst=0;
            return tr_out;
        end
        and_op: begin
            tr_out.result = tr_in.a & tr_in.b;
            tr_out.result_rst=0;
            return tr_out;
        end
        xor_op: begin
            tr_out.result = tr_in.a ^ tr_in.b;
            tr_out.result_rst=0;
            return tr_out;
        end
        mul_op: begin
            tr_out.result = tr_in.a * tr_in.b;
            tr_out.result_rst=0;
            return tr_out;
        end
        rst_op: begin
            tr_out.result ='0;
            tr_out.result_rst=1;
            return tr_out;
        end
    endcase

    return null;



endfunction
```

Figure 6: Arithmetic and software-reset reference calculation.

## 10 Scoreboard ordering and legal cancellation

The scoreboard consumes expected and actual records in order. A complete ordinary result must match both numeric result and reset flag. A clean reset result must match the reset reference. A mismatch increments the failed count and prints both objects; a nonzero failed count terminates the test as a failure.

When an actual reset arrives while the next expected record is an arithmetic result, the reset legally cancels that pending operation. The observable result must still be zero. The scoreboard increments a separate aborted counter and consumes the following reset reference to check the actual reset notification. It does not count the canceled arithmetic operation as a passed result. Unexpected reset behavior cannot silently pass because the following expected record must also compare as a reset.

Output functional coverage is sampled after this classification, so interruption coverage includes a reset before the first result beat as well as during result collection. The canceled operation and completed reset command are represented separately for coverage accounting without placing duplicate physical output records in the mailbox. The final summary reports passed, aborted and failed counts, and both remaining queue sizes. The automated runner requires zero failures and no pending records.

## 11 Testbench integration and test profiles

The top-level testbench starts with physical reset asserted, releases it after 200 ns, and generates a clock that toggles every 5 ns, for a 10 ns period. Interfaces and the DUT share that clock. Compile-time test selection chooses random_test, test_mid, test_high or test_mix. The selected name is also recorded in the coverage database so that labels agree with the instantiated test.

random_test uses the complete legal operand space. test_mid uses explicit width-safe bounds from the midpoint through the maximum unsigned operand. test_high restricts operands to the largest two representable values. test_mix successively uses unrestricted, mid-range, high-range and low-range profiles. Range endpoints are inclusive. A selected blueprint is installed before generation begins. Reset/no-op operand defaults are not counted as numeric operand coverage.

The mid and high tests run for 20000 ns; the mixed profile runs through its successive timed phases, and the random test runs for approximately 100000 ns. Each environment includes generator, driver, model, input monitor, output monitor and scoreboard threads. Tests terminate with a scoreboard summary rather than relying on an infinite background clock to stop by itself.

```
1
2  alu_multi_cycle #(
3      .OPERAND_BUS_WIDTH(OPERAND_BUS_WIDTH),
4      .OPERAND_MAX_DATA_WIDTH(OPERAND_MAX_DATA_WIDTH),
5      .RESULT_BUS_WIDTH(RESULT_BUS_WIDTH),
6      .RESULT_MAX_DATA_WIDTH(RESULT_MAX_DATA_WIDTH)
7  ) dut (
8      .clk    (in_bus.clk),
9      .rst    (rst),
10     .ready  (in_bus.ready),
11     .operand_last (in_bus.operand_last),
12     .operand_valid  (in_bus.operand_valid),
13     .op     (in_bus.op),
14     .a      (in_bus.a),
15     .b      (in_bus.b),
16     .result_valid   (out_bus.result_valid),
17     .result_last   (out_bus.result_last),
18     .result_rst   (out_bus.result_rst),
19     .result (out_bus.result)
20 );
```

Figure 7: Width parameters and DUT port connections.

## 12 Functional coverage and complete test plan

The input operation coverpoint has six legal bins and default illegal bins for undefined opcodes. Each legal bin requires at least ten hits. Operand coverpoints are sampled only for add, AND, XOR and multiply operations. For 32-bit unsigned operands, low covers zero through 0x7fffffff, mid covers 0x80000000 through 0xfffffffd, and high covers 0xfffffffe through 0xffffffff. These three ranges are disjoint and include both endpoint values.

The idle-gap coverpoint excludes reset commands and has three bins: zero, one through eight, and nine through fourteen. The three groups match the constrained input domain. Fifteen is not a legal requested gap. The input group contains fifteen bins: six operation bins, three a bins, three b bins and three gap bins.

The output numeric result is covered only when the record is neither a reset nor an aborted operation. Its two bins span zero through 0x7fffffffffffff and 0x8000000000000000 through 0xffffffffffffff. result_rst has zero and one bins. reset_during_calculation has noninterrupted and interrupted bins. The output group contains six bins, giving twenty-one defined functional bins in total. Each bin uses an at-least-ten hit goal.

The test-plan CSV links operation, a, b, gap, result, reset notification and interruption coverpoints. The input section has weight three, the output section weight one, and the operation requirement weight five. Every requirement's goal is 100 percent. The supplied workbook representation and the native plan table preserve all of these links and weights. Coverage measures exercised definitions, not the correctness of every possible input or the completeness of the definitions themselves.

Table 2: Complete spreadsheet-style coverage plan

| Requirement | Link suffix | Weight | Goal |
|---|---|---|---|
| Input section | - | 3 | 100% |
| Operations | ALU_in_transaction_cg:op | 5 | 100% |
| Operand A | ALU_in_transaction_cg:a | 1 | 100% |
| Operand B | ALU_in_transaction_cg:b | 1 | 100% |
| Clock gap | ALU_in_transaction_cg:clk_gap | 1 | 100% |
| Output section | - | 1 | 100% |
| Result | ALU_out_transaction_cg:result | 1 | 100% |
| Reset observed | ALU_out_transaction_cg:result_rst | 1 | 100% |
| Reset interruption | ALU_out_transaction_cg:reset_during_calculation | 1 | 100% |

| G | F | E | D | C | B | A |
|---|---|---|---|---|---|---|
| Goal | Weig | Type | Link | Description | Title | Section |
| 100 | 3 | | | alu in transaction coverage | alu_in | 1 |
| 100 | 5 | CoverPoint | ALU_in_transaction_cg:op | this cover point tracks the available ALU operations | operations | 1.1 |
| 100 | 1 | CoverPoint | ALU_in_transaction_cg:a | this cover point tracks the Operand A | Operand A | 1.2 |
| 100 | 1 | CoverPoint | ALU_in_transaction_cg:b | this cover point tracks the Operand B | Operand B | 1.3 |
| 100 | 1 | CoverPoint | ALU_in_transaction_cg:clk_gap | this cover point tracks the gap between operatins | clock gap | 1.4 |
| 100 | 1 | | | alu out transaction coverage | alu out | 2 |
| 100 | 1 | coverpoint | ALU_out_transaction_cg:result | this cover point tracks the Result | New Section 2.1 | 2.1 |
| 100 | 1 | coverpoint | ALU_out_transaction_cg:result_rst | this cover point tracks the Result_rst | rst op happenning | 2.2 |
| 100 | 1 | coverpoint | ALU_out_transaction_cg:reset_during_calculation | this cover point tracks the operation rst of the ALU during data gathering | rst op during getting output | 2.3 |

Figure 8: Spreadsheet representation in the recorded academic report.

## 13 Recorded academic coverage and ranking figures

The recorded classroom report contains a Questa HTML summary with two tests marked OK, a weighted test-plan total of 100 percent, and twenty-one bins shown as covered. The output result, result_rst and reset_during_calculation displays each show two covered bins and no missing bins. These figures are retained as the academic experiment record, together with the original spreadsheet and rank summaries.

One captured ranking lists test_mid42 with total coverage 88.06 percent and test-plan coverage 93.75 percent. The other lists test_high42 with total coverage 87.46 percent and test-plan coverage 100 percent. Both captured summaries report 0.06 seconds of CPU time; their simulation-time entries are 20050000 ps and 20000000 ps respectively. These are different reported metrics and captured views. A 100-percent test-plan goal is not the same quantity as the total-coverage percentage, and the two screenshots do not demonstrate monotonic aggregate coverage between two database merges.

Zero operand values associated with reset/no-op transactions do not establish that a numeric low-range test has been exercised. The arithmetic operand coverpoints therefore use the operation qualification described above. The ranking objective is to identify a small useful set of tests against defined coverage goals, rather than infer correctness from a high percentage alone.

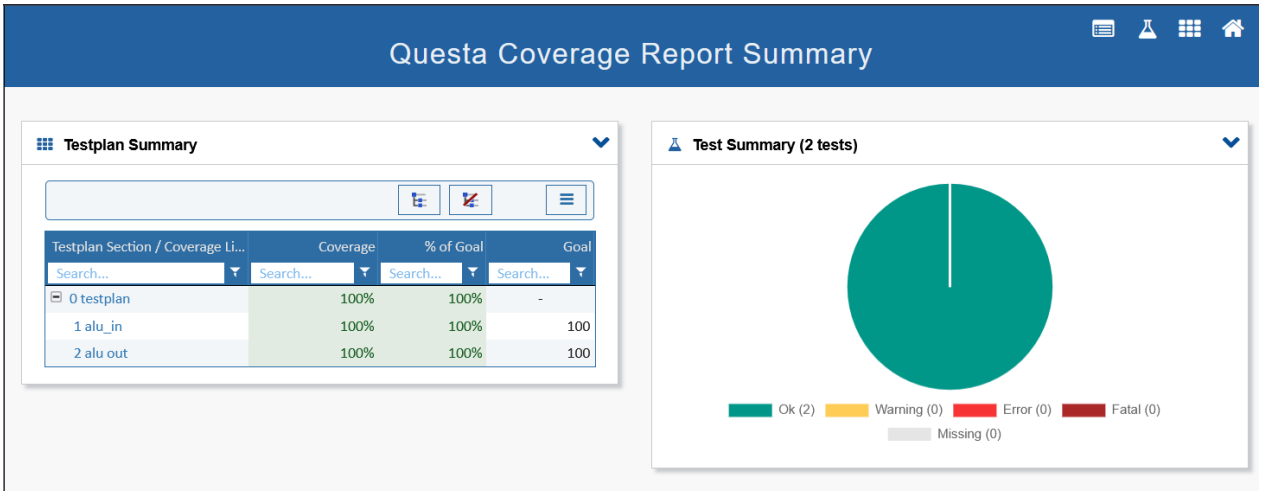



Figure 9: Recorded Questa test-plan summary and two-test status.

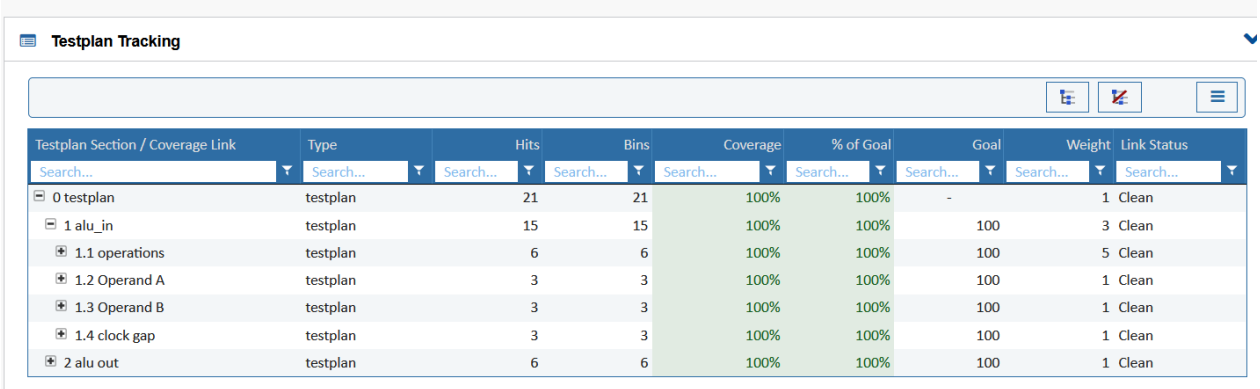



Figure 10: Recorded test-plan tracking view with twenty-one functional bins.

coverpoints

| Coverpoints | Bins | Hits | Misses | Excluded | Goal | Coverage |
|---|---|---|---|---|---|---|
| result | 2 | 2 | 0 | 0 | 100 | 100% |
| result_rst | 2 | 2 | 0 | 0 | 100 | 100% |
| reset_during_calculation | 2 | 2 | 0 | 0 | 100 | 100% |

Figure 11: Recorded output coverpoints: result, reset and interruption.

```
Rank   TotalCov  Testname
-----  --------  --------------------
    1     88.06  test_mid42
All 1 items have been ranked.

Ranking summary:
      Total Contributing Tests      = 1
      Total Non-Contributing Tests  = 0
      Total Coverage                = 88.06%
      Testplan Coverage             = 93.75%
      Total CPU Time                = 0.06 s
      Total SIM Time                = 20050000.00 ps
```

Figure 12: Captured academic ranking output for test_mid42.

```
Rank   TotalCov  Testname
-----  --------  --------------------
    1     87.46  test_high42
No further improvement to coverage.

Ranking summary:
      Total Contributing Tests      = 1
      Total Non-Contributing Tests  = 1
      Total Coverage                = 87.46%
      Testplan Coverage             = 100.00%
      Total CPU Time                = 0.06 s
      Total SIM Time                = 20000000.00 ps
```

Figure 13: Captured academic ranking output for test_high42.

## 14 Reproducible simulation results

The distributed runner compiles and elaborates each selected test using Questa 2021.1, runs it with seed 42, captures the scoreboard and saves its UCDB. Compilation and elaboration report zero errors and zero warnings for the four test profiles. The results table gives completed comparisons, legal cancellations, failures, queue sizes and functional covergroup coverage for each execution.

All four runs have zero failed comparisons and empty expected/actual queues at the end. Together they contain 1164 completed comparisons and 189 separately classified legal cancellations. Merging their functional coverage covers all twenty-one defined bins and reports 100 percent covergroup coverage. This statement is limited to the explicit functional model and hit goals; it is not a claim of complete RTL code coverage or a formal proof.

Iterative functional-coverage ranking uses the four individual databases. random_test42 contributes a reported aggregate functional metric of 91.66 percent; adding test_high42 reaches 100 percent. Two tests contribute and two do not add coverage under that ranking goal. The ranking summary reports 0.02 seconds of CPU time and 120050000 ps of simulation time. The decimal value printed by the ranking tool is retained rather than interpreted as an exact count fraction.

Table 3: Seed-42 simulation summaries and functional covergroup coverage

| Test | Passed | Aborted | Failed | Queues E/A | Functional |
|---|---|---|---|---|---|
| test_mid | 132 | 18 | 0 | 0/0 | 83.33% |
| test_high | 133 | 19 | 0 | 0/0 | 83.33% |
| test_mix | 235 | 39 | 0 | 0/0 | 87.50% |
| random_test | 664 | 113 | 0 | 0/0 | 91.66% |

Table 4: Iterative functional ranking from the individual databases

| Rank | Test | Functional metric |
|---|---|---|
| 1 | random_test42 | 91.66% |
| 2 | test_high42 | 100.00% |

# 15 Execution and engineering limits

run.py locates Questa tools on PATH, creates a separate build library and results directory for each test/seed, compiles sources in dependency order, elaborates top, and loads the elaborated design directly in command-line mode. Relative library paths avoid legacy simulator parsing of spaces in a Windows parent directory. The runner checks diagnostic messages as well as process return codes, because a simulator can finish with a successful process code after reporting a fatal testbench failure.

The package includes the full RTL, interfaces, class environment, tests, test plan, compiled-result summaries, UCDBs and functional-coverage/ranking output. The build libraries are regenerable and are not required to inspect the project. The report does not establish FPGA placement, timing closure, power or area improvement. Verification uses a finite test set and a model; additional assertions, directed protocol cases and formal methods could expand assurance beyond the stated experiments.

# A Implementation details

## A.1 ALU_out_transaction.sv

```
//------------------------------------
// Last Update  : 1403/3
// Author       : Alireza Abbasian
//------------------------------------

class ALU_out_transaction;

//-------------------The random variables

//------------------------------------
    time issue_time;
    bit [RESULT_MAX_DATA_WIDTH-1:0] result ;
    bit result_rst;
    bit aborted;

//---------------------------constraints

//------------------------------------

//-----------------------------functions
    function new();

        this.issue_time = $time;
        this.aborted=0;
    endfunction

    virtual function string convert2string();

    string string_out = "ALU_out_transaction:\n";

        string_out = {string_out, $sformatf("result:0x%x \n",result)};
        string_out = {string_out, $sformatf("result_rst= %b\n",
    result_rst)};

        return string_out;

    endfunction

    virtual function void do_print();

        $display(convert2string());

    endfunction

    virtual function bit do_compare (input ALU_out_transaction rhs);
```

Figure 14: Implementation: ALU_out_transaction.sv (panel 1 of 2).

```
        return (
                  (this.result == rhs.result) && (this.result_rst ==
    rhs.result_rst)
                );

    endfunction

    virtual function ALU_out_transaction do_copy();
        ALU_out_transaction dst;

        dst = new();

        dst.result  = this.result;
        dst.result_rst  = this.result_rst;
        dst.aborted  = this.aborted;
        return dst;

    endfunction : do_copy

//-------------------------------------

endclass
```

Figure 15: Implementation: ALU_out_transaction.sv (panel 2 of 2).

## A.2 ALU_in_transaction_range.sv

```
//---------------------------------------
// Last Update  : 1403/3
// Author       : Alireza Abbasian
//---------------------------------------

class ALU_in_transaction_range extends ALU_in_transaction;

//---------------------------------------
     bit [OPERAND_MAX_DATA_WIDTH-1:0] a_min;
     bit [OPERAND_MAX_DATA_WIDTH-1:0] a_max;
     bit [OPERAND_MAX_DATA_WIDTH-1:0] b_min;
     bit [OPERAND_MAX_DATA_WIDTH-1:0] b_max;

//---------------------------constraints

    constraint a_range {
        a >=    a_min;
        a <=     a_max;
    }

    constraint b_range {
        b >=    b_min;
        b <=     b_max;
    }

//---------------------------------------

//-----------------------------functions
    function new();

        super.new();

    endfunction

    virtual function string convert2string();

        string string_out = super.convert2string();

        string_out = {
            string_out,
            $sformatf(
                "ranged transaction: a in [%0d,%0d], b in [%0d,%0d]\n",
                a_min, a_max, b_min, b_max
            )
        };
```

Figure 16: Implementation: ALU_in_transaction_range.sv (panel 1 of 2).

```
        return string_out;

    endfunction

    virtual function void do_print();

        $display(convert2string());

    endfunction

    virtual function bit do_compare (input ALU_in_transaction rhs);
        ALU_in_transaction_range rhs_range;

        // Ensure the cast is valid before proceeding
        if (!$cast(rhs_range, rhs)) begin
            return super.do_compare(rhs);
        end

        return super.do_compare(rhs_range) &&
               (this.a_min == rhs_range.a_min) &&
               (this.a_max == rhs_range.a_max) &&
               (this.b_min == rhs_range.b_min) &&
               (this.b_max == rhs_range.b_max);
    endfunction

    virtual function ALU_in_transaction do_copy();
        ALU_in_transaction_range dst;

        dst = new();

        dst.op     = this.op;
        dst.a      = this.a;
        dst.b      = this.b;
        dst.clk_gap = this.clk_gap;

        dst.a_min  = this.a_min;
        dst.a_max  = this.a_max;
        dst.b_min  = this.b_min;
        dst.b_max  = this.b_max;

        return dst;
    endfunction : do_copy

//---------------------------------------

endclass
```

Figure 17: Implementation: ALU_in_transaction_range.sv (panel 2 of 2).

## A.3 ALU_in_driver.sv

```
// Supplied RTL / framework attribution: Alireza Abbasian.
// Verification project: Reza Sanaee Nezhad.

class ALU_in_driver;
    mailbox gen2drv;
    event drv2gen;
    virtual ALU_in_if in_bus;
    virtual ALU_out_if out_bus;
    function new(mailbox gen2drv,event drv2gen,virtual ALU_in_if.driver_mp
    in_bus,
                 virtual ALU_out_if.monitor_mp out_bus);
        this.gen2drv=gen2drv;this.drv2gen=drv2gen;this.in_bus=in_bus;this.ou
    t_bus=out_bus;
    endfunction
    task run();
        ALU_in_transaction tr;
        in_bus.operand_valid=0;in_bus.operand_last=0;in_bus.op=no_op;in_bus.
    a=0;in_bus.b=0;
        forever begin gen2drv.get(tr);send(tr);->drv2gen;end
    endtask
    task send(ALU_in_transaction tr);
        localparam int NUM_BEATS=OPERAND_MAX_DATA_WIDTH/OPERAND_BUS_WIDTH;
        // Drive on falling edges; the DUT accepts stable data on rising
    edges.
        if(in_bus.clk !== 1'b0) @(negedge in_bus.clk);
        while(!in_bus.rst || (!in_bus.ready && tr.op!=rst_op)) @(negedge
    in_bus.clk);
        if(tr.op==rst_op) begin
            in_bus.op=rst_op;in_bus.operand_valid=1;in_bus.operand_last=1;in
    _bus.a=0;in_bus.b=0;
            @(posedge in_bus.clk);@(negedge in_bus.clk);
        end else begin
            repeat(tr.clk_gap) @(negedge in_bus.clk);
            for(int i=0;i<NUM_BEATS;i++) begin

    in_bus.operand_valid=1;in_bus.operand_last=(i==NUM_BEATS-
    1);in_bus.op=tr.op;
                in_bus.a=tr.a[i*OPERAND_BUS_WIDTH+:OPERAND_BUS_WIDTH];
                in_bus.b=tr.b[i*OPERAND_BUS_WIDTH+:OPERAND_BUS_WIDTH];
                @(posedge in_bus.clk);@(negedge in_bus.clk);
            end
        end
        in_bus.operand_valid=0;in_bus.operand_last=0;in_bus.op=no_op;
    endtask
endclass
```

Figure 18: Implementation: ALU_in_driver.sv.

## A.4 ALU_in_monitor.sv

```
// Supplied RTL / framework attribution: Alireza Abbasian.
// Verification project: Reza Sanaee Nezhad.

class ALU_in_monitor;
    virtual ALU_in_if bus;
    mailbox mon2env;
    ALU_in_coverage monitor_coverage;
    function new(virtual ALU_in_if.monitor_mp bus,mailbox mon2env);
        this.bus=bus;this.mon2env=mon2env;monitor_coverage=new();
    endfunction
    task run();
        ALU_in_transaction tr;
        forever begin
    do_monitor(tr);monitor_coverage.sample(tr);mon2env.put(tr.do_copy());end
    endtask
    task do_monitor(output ALU_in_transaction tr);
        int idle_cycles=0,beat=0;
        forever begin
            @(posedge bus.clk);
            if(!bus.rst) idle_cycles=0;
            else if(bus.operand_valid && (bus.ready || bus.op==rst_op))
    break;
            else if(bus.ready) idle_cycles++;
            else idle_cycles=0;
        end
        tr=new();tr.op=alu_in_op_t'(bus.op);
        if(tr.op==rst_op) return;
        if(idle_cycles>14) $fatal(1,"Observed idle gap exceeds the stimulus
    contract: %0d",idle_cycles);
        tr.clk_gap=idle_cycles;
        forever begin
            if(bus.operand_valid && bus.ready) begin
                if(beat>=OPERAND_MAX_DATA_WIDTH/OPERAND_BUS_WIDTH)
    $fatal(1,"Too many operand beats");
                tr.a[beat*OPERAND_BUS_WIDTH+:OPERAND_BUS_WIDTH]=bus.a;
                tr.b[beat*OPERAND_BUS_WIDTH+:OPERAND_BUS_WIDTH]=bus.b;
                beat++;
                if(bus.operand_last) break;
            end
            @(posedge bus.clk);
            if(!bus.rst || (bus.operand_valid && bus.op==rst_op))
    $fatal(1,"Input frame interrupted before completion");
        end
    endtask
endclass
```

Figure 19: Implementation: ALU_in_monitor.sv.

## A.5 ALU_out_monitor.sv

```
// Supplied RTL / framework attribution: Alireza Abbasian.
// Verification project: Reza Sanaee Nezhad.

class ALU_out_monitor;
    virtual ALU_out_if bus;
    mailbox mon2env;
    ALU_out_coverage monitor_coverage;
    function new(virtual ALU_out_if.monitor_mp bus,mailbox mon2env);
        this.bus=bus;this.mon2env=mon2env;monitor_coverage=new();
    endfunction
    task run();
        ALU_out_transaction tr;
        forever begin do_monitor(tr);mon2env.put(tr.do_copy());end
    endtask
    task do_monitor(output ALU_out_transaction tr);
        int beat=0;
        @(negedge bus.clk);
        while(!bus.rst || (!bus.result_valid && !bus.result_rst)) @(negedge
    bus.clk);
        tr=new();tr.result=0;
        if(bus.result_rst) begin
            tr.result=bus.result;tr.result_rst=1;
        end else begin
            forever begin
                if(bus.result_rst) begin
                    tr.result=0;tr.result_rst=1;tr.aborted=1;break;
                end
                if(!bus.result_valid) $fatal(1,"Result frame ended without
    result_last or reset");
                if(beat>=RESULT_MAX_DATA_WIDTH/RESULT_BUS_WIDTH)
    $fatal(1,"Too many result beats");

    tr.result[beat*RESULT_BUS_WIDTH+:RESULT_BUS_WIDTH]=bus.result;beat++;
                if(bus.result_last) break;
                @(negedge bus.clk);
            end
        end
        // Each active software-reset cycle corresponds to one accepted
    reset command.
        // The next call starts at the next falling edge, including back-to-
    back resets.
    endtask
endclass
```

Figure 20: Implementation: ALU_out_monitor.sv.

## A.6 Scoreboard.sv

```
// Supplied framework attribution: Alireza Abbasian.
// Verification project: Reza Sanaee Nezhad.
class Scoreboard;
    mailbox model2scb,mon_out_2_scb;
    ALU_out_coverage monitor_coverage;
    int failed_num=0,passed_num=0,aborted_num=0;
    function new(mailbox model2scb,mailbox mon_out_2_scb);
        this.model2scb=model2scb;this.mon_out_2_scb=mon_out_2_scb;
        monitor_coverage=new();
    endfunction
    task run();
        ALU_out_transaction expected,actual,reset_expected,reset_actual;
        forever begin
            model2scb.get(expected);mon_out_2_scb.get(actual);
            if(actual.result_rst && !expected.result_rst) begin
                // A reset cancels the pending operation, even before its
    first output beat.
                if(actual.result!=0) begin failed_num++;$display("Reset
    produced nonzero output");end
                else aborted_num++;
                actual.aborted=1;monitor_coverage.sample(actual);
                model2scb.get(reset_expected);
                reset_actual=actual.do_copy();reset_actual.aborted=0;
                check_actual(reset_expected,reset_actual);
                monitor_coverage.sample(reset_actual);
            end else begin

    check_actual(expected,actual);monitor_coverage.sample(actual);
            end
        end
    endtask
    function void check_actual(ALU_out_transaction
    expected,ALU_out_transaction actual);
        if(!actual.aborted && expected.do_compare(actual)) passed_num++;
        else begin failed_num++;$display("Mismatch at
    %0t",$time);expected.do_print();actual.do_print();end
    endfunction
    function void wrap_up();
        $display("SCOREBOARD: passed=%0d aborted=%0d failed=%0d
    expected_pending=%0d actual_pending=%0d",

    passed_num,aborted_num,failed_num,model2scb.num(),mon_out_2_scb.num());
        if(failed_num) $fatal(1,"Scoreboard failure");
        $finish;
    endfunction
endclass
```

Figure 21: Implementation: Scoreboard.sv.

## A.7 ALU_in_coverage.sv

```
//---------------------------------------
// Last Update  : 1403/3
// Author       : Alireza Abbasian
//---------------------------------------

class ALU_in_coverage;

    localparam int  N       = OPERAND_MAX_DATA_WIDTH;

    localparam bit [N-1:0] MAX_VAL    = {N{1'b1}};
    localparam bit [N-1:0] MID_RANGE  = MAX_VAL >> 1;
    localparam bit [N-1:0] HIGH_RANGE = MAX_VAL;

    covergroup ALU_in_transaction_cg with function sample(ALU_in_transaction
     tr_in);

        option.auto_bin_max=64;

        op: coverpoint tr_in.op {
            option.at_least = 10;
            bins noop   = { 3'b000 };
            bins addop  = { 3'b001 };
            bins andop  = { 3'b010 };
            bins xorop  = { 3'b011 };
            bins mulop  = { 3'b100 };
            bins rstop  = { 3'b111 };
            illegal_bins invalid_op = default;
        }

        a: coverpoint tr_in.a iff (tr_in.op inside
    {add_op,and_op,xor_op,mul_op}) {
            option.at_least = 10;
            bins low    = {   [0:MID_RANGE] };
            bins mid    = { [MID_RANGE+1:HIGH_RANGE-2] };
            bins high   = { [HIGH_RANGE-1:HIGH_RANGE] };
        }

        b: coverpoint tr_in.b iff (tr_in.op inside
    {add_op,and_op,xor_op,mul_op}) {
            option.at_least = 10;
            bins low    = {   [0:MID_RANGE] };
            bins mid    = { [MID_RANGE+1:HIGH_RANGE-2] };
            bins high   = { [HIGH_RANGE-1:HIGH_RANGE] };
        }

        clk_gap: coverpoint tr_in.clk_gap iff (tr_in.op != rst_op) {
```

Figure 22: Implementation: ALU_in_coverage.sv (panel 1 of 2).

```
            option.at_least = 10;
            bins zero    = { 0 };
            bins Low     = { [1:8] };
            bins high    = { [9:14]};
        }

    endgroup

    function new();
        ALU_in_transaction_cg = new();
    endfunction

    function void sample(ALU_in_transaction tr_in);

        ALU_in_transaction_cg.sample(tr_in);
    endfunction

endclass
```

Figure 23: Implementation: ALU_in_coverage.sv (panel 2 of 2).

## A.8 ALU_out_coverage.sv

```
//---------------------------------------
// Last Update  : 1403/3
// Author       : Alireza Abbasian
//---------------------------------------

class ALU_out_coverage;

    localparam int  N       = RESULT_MAX_DATA_WIDTH;

    localparam bit [N-1:0] MAX_VAL    = {N{1'b1}};
    localparam bit [N-1:0] MID_RANGE  = MAX_VAL >> 1;
    localparam bit [N-1:0] HIGH_RANGE = MAX_VAL;

    covergroup ALU_out_transaction_cg with function
    sample(ALU_out_transaction tr_out);

        option.auto_bin_max=64;

        result: coverpoint tr_out.result iff (!tr_out.result_rst &&
    !tr_out.aborted) {
            option.at_least = 10;
            bins low    = {   [0:MID_RANGE] };
            bins high    = { [MID_RANGE+1:HIGH_RANGE] };
        }

        result_rst: coverpoint tr_out.result_rst {
            option.at_least = 10;
            bins low    = { 0 };
            bins high    = { 1 };
        }

        reset_during_calculation: coverpoint tr_out.aborted {
            option.at_least = 10;
            bins NO_mid_cal_reset    = { 0 };
            bins mid_cal_reset    = { 1 };
        }

    endgroup

    function new();
        ALU_out_transaction_cg = new();
    endfunction

    function void sample(ALU_out_transaction tr_out);

        ALU_out_transaction_cg.sample(tr_out);
```

Figure 24: Implementation: ALU_out_coverage.sv (panel 1 of 2).

```
    endfunction

endclass
```

Figure 25: Implementation: ALU_out_coverage.sv (panel 2 of 2).

## A.9 alu_multi_cycle.sv

```
//-------------------------------------
// Last Update  : 1403/3
// Author       : Alireza Abbasian
//-------------------------------------

module alu_multi_cycle #(
    parameter OPERAND_BUS_WIDTH = 8,
    parameter OPERAND_MAX_DATA_WIDTH = 32,
    parameter RESULT_BUS_WIDTH = 16,
    parameter RESULT_MAX_DATA_WIDTH = 64
) (
    input   wire                            clk,
    input   wire                            rst, // active low
    input   wire                            operand_valid,
    input   wire [2:0]                      op,
    input   wire [OPERAND_BUS_WIDTH-1:0]    a,
    input   wire [OPERAND_BUS_WIDTH-1:0]    b,
    input   wire                            operand_last,
    output  reg                             ready,
    output  reg                             result_valid,
    output  reg [RESULT_BUS_WIDTH-1:0]      result,
    output  reg                             result_last,
    output  reg                             result_rst
);

    // no_op  = 3'b000,
    // add_op = 3'b001,
    // and_op = 3'b010,
    // xor_op = 3'b011,
    // mul_op = 3'b100,
    // rst_op = 3'b111

    typedef enum logic [1:0] {
        IDLE,
        RECEIVE,
        EXECUTE,
        SEND
    } state_t;

    state_t state, next_state;

    reg [OPERAND_MAX_DATA_WIDTH-1:0] operand_a, operand_b;
    reg [RESULT_MAX_DATA_WIDTH-1:0] result_full;

    reg [$clog2(OPERAND_MAX_DATA_WIDTH/OPERAND_BUS_WIDTH):0]
    operand_shift_cnt;
```

Figure 26: Implementation: alu_multi_cycle.sv (panel 1 of 4).

```
reg [$clog2(RESULT_MAX_DATA_WIDTH/RESULT_BUS_WIDTH):0] result_shift_cnt;

reg [2:0] op_reg;
reg [RESULT_MAX_DATA_WIDTH-1:0] calculated_result;
initial begin
    if (OPERAND_BUS_WIDTH<=0 || RESULT_BUS_WIDTH<=0 ||
        OPERAND_MAX_DATA_WIDTH%OPERAND_BUS_WIDTH ||
RESULT_MAX_DATA_WIDTH%RESULT_BUS_WIDTH ||
        RESULT_MAX_DATA_WIDTH<2*OPERAND_MAX_DATA_WIDTH)
        $fatal(1,"Unsupported ALU width parameters");
end
always @(*) begin
    case(op_reg)
        3'b001: calculated_result = operand_a + operand_b;
        3'b010: calculated_result = operand_a & operand_b;
        3'b011: calculated_result = operand_a ^ operand_b;
        3'b100: calculated_result = operand_a * operand_b;
        default: calculated_result = 0;
    endcase
end

always @( * ) begin
    result = result_full[result_shift_cnt*RESULT_BUS_WIDTH +:
RESULT_BUS_WIDTH];

    ready = state == IDLE || state == RECEIVE;
end

// Main logic
always @(posedge clk) begin

    if (op == 3'b111 && operand_valid || (!rst)) begin

        state <= IDLE;
        result_rst          <= 1'b1;

        result_valid        <= 1'b0;

        result_last         <= 1'b0;
        operand_a           <= '0;
        operand_b           <= '0;
        result_full         <= '0;
        operand_shift_cnt   <= 0;
        result_shift_cnt    <= 0;
        op_reg              <= 3'b000;
```

Figure 27: Implementation: alu_multi_cycle.sv (panel 2 of 4).

```
    end else begin

        result_rst <= 1'b0;

        case (state)

            IDLE: begin

                operand_shift_cnt <= 0;
                operand_a <= '0;
                operand_b <= '0;
                result_valid <= 0;
                result_last <= 0;
                result_shift_cnt <= 0;
                result_full         <= '0;

                if (operand_valid) begin
                    operand_a[OPERAND_BUS_WIDTH-1:0] <= a;
                    operand_b[OPERAND_BUS_WIDTH-1:0] <= b;
                    op_reg <= op;

                    operand_shift_cnt <= 1;

                    if (op == 0) begin
                        state <= IDLE;
                    end else if (operand_last) begin
                        state <= EXECUTE;
                    end else begin
                        state <= RECEIVE;
                    end

                end

            end
            RECEIVE: begin

                if (operand_valid) begin

if(operand_shift_cnt>=OPERAND_MAX_DATA_WIDTH/OPERAND_BUS_WIDTH)
                        $fatal(1,"Operand frame exceeds declared
width");
                    operand_a[operand_shift_cnt*OPERAND_BUS_WIDTH +:
OPERAND_BUS_WIDTH] <= a;
                    operand_b[operand_shift_cnt*OPERAND_BUS_WIDTH +:
OPERAND_BUS_WIDTH] <= b;
                    operand_shift_cnt <= operand_shift_cnt + 1;
```

Figure 28: Implementation: alu_multi_cycle.sv (panel 3 of 4).

```
                    if (operand_last) begin

                        state <= EXECUTE;
                    end
                end

            end
            EXECUTE: begin

                result_full <= calculated_result;

                result_shift_cnt <= 0;
                result_valid <= 1;

                if ((calculated_result >> RESULT_BUS_WIDTH) == 0) begin
                    state <= IDLE;
                    result_last <= 1;
                end else begin
                    state <= SEND;
                end

            end
            SEND: begin

                result_shift_cnt <= result_shift_cnt + 1;
                result_valid <= 1;

                if (result_full >> (result_shift_cnt+2)*RESULT_BUS_WIDTH
     == 0) begin
                    state <= IDLE;
                    result_last <= 1;
                end

            end
        endcase
      end
    end

endmodule
```

Figure 29: Implementation: alu_multi_cycle.sv (panel 4 of 4).